

Reproduction guide

Written for someone starting from an empty directory. Every command below was run to produce the numbers in [report/changelog.md](#).

There are two halves, and the first one needs no cloud account and no money:

- **The harness** — the fixture stack, the plans, the scanner baselines and the scorer. Terraform and Python only. About 6 minutes, \$0.
- **The model runs** — the baselines and the six iterations. Needs AWS Bedrock. About 3 hours and \$10 to reproduce everything, or 4 minutes and \$0.11 for the single result that matters.

If you only have a few minutes, do [the short path](#).

Versions

These are the versions the results were produced with. Nothing here is version-fragile except the AWS provider, which is pinned by `infra/base/.terraform.lock.hcl`.

	version
Terraform	1.12.2
AWS provider	5.100.0 (pinned in the lock file)
Python	3.12.10
checkov	3.3.15
anthropic	1.2.0
boto3	1.35.49
OS used	Windows 11; the shell scripts need bash (Git Bash is fine)

```
git clone https://github.com/Jamesokooboh/blast-radius && cd blast-radius
pip install -r requirements.txt
terraform -version      # 1.9+ required
```

Terraform needs network access exactly once, to download the AWS provider.

Part 1 — the harness (no AWS account, no cost)

Build every plan and diff

```
./plan.sh --all
```

~4 minutes. For each of the 15 cases this copies the base stack, applies the case's overlay, plans it against the committed state fixture, and writes `results/plans/<case>.plan.json` and `results/diffs/<case>.diff`.

Expected output — every case produces the plan signal it was designed for:

```
--- case base ---      (no changes)
--- case 08 ---        delete/create  aws_db_instance.main
--- case 12 ---        update         aws_iam_role_policy.app_data
--- case 13 ---        (no changes)
```

base must be a clean no-op. Case 08 must show delete/create — that is the production database being replaced, and several results depend on it existing. Case 12 must show the IAM policy changing even though its diff touches only `variables.tf`.

No AWS credentials are used. The provider in `infra/base/versions.tf` is configured with the literal key `test` and every call that would reach the API disabled. See [the state fixture](#) for why this works.

Run the scanner baselines

```
python tools/run_checkov.py --all          # ~6 min: what CI runs today
python tools/run_checkov.py --all --scoped # ~6 min: scoped to changed resources
```

Checkov is slow — about 20 seconds per case, twice over.

Score everything

```
python score.py --mode checkov          # expect F1 0.052
python score.py --mode checkov-scoped   # expect F1 0.320
```

Confirm the scorer works in both directions

```
python tools/make_oracle.py && python score.py --mode oracle # expect F1 1.000
python score.py --mode nonexistent-mode                     # expect F1 0.000
```

`make_oracle.py` replays the labels back as findings. It is a round-trip test, not a measurement: it shows a perfect answer scores 1.000 and an empty one scores 0.000, so the scorer discriminates rather than being pinned at one end.

Self-checks

```
python score.py --selfcheck          # scoring logic, including address normalisation
python tools/planfilter.py           # the phantom-change filter
python tools/pricing.py              # the price table
```

Part 2 — the model runs (needs AWS Bedrock)

What you need

An AWS account with Bedrock access to `us.anthropic.claude-sonnet-4-6` and `us.anthropic.claude-haiku-4-5-20251001-v1:0` in `us-east-1`, and ordinary AWS credentials on the machine. There is no Anthropic API key anywhere in this project; authentication is the standard AWS credential chain.

```
aws sts get-caller-identity          # confirm which account you are about to bill
export AWS_PROFILE=your-profile      # tools/model.py defaults to the profile "Joseph"
```

Every runner prints the profile, account and model before its first billable call, and records the account id in each trajectory.

Three things that cost time when setting this up, recorded so you skip them:

1. Bedrock serves these models through **inference profiles**. The id needs a `us.` prefix and must go through `AnthropicBedrock` (the `bedrock-runtime` `InvokeModel` client), not `AnthropicBedrockMantle`. `anthropic.claude-sonnet-5` returns 403; `us.anthropic.claude-sonnet-5` returns 404 on one endpoint and

AccessDenied on the other; `us.anthropic.claude-sonnet-4-6` works. 2. `ListFoundationModels` returns ids for the *other* endpoint — dated forms like `anthropic.claude-haiku-4-5-20251001-v1`, which 404 here. Do not copy ids from that listing. 3. The Bedrock console's "Model access" page has been retired. Access is granted per account and per region and may simply not be available to yours; the Claude 5 family was AccessDenied on both accounts tested.

Check what your account can actually call before spending anything:

```
python - <<'EOF'
import os, anthropic
from anthropic import AnthropicBedrock
c = AnthropicBedrock(aws_region="us-east-1")
for m in ["us.anthropic.claude-sonnet-4-6",
         "us.anthropic.claude-haiku-4-5-20251001-v1:0"]:
    try:
        c.messages.create(model=m, max_tokens=4,
                          messages=[{"role": "user", "content": "hi"}])
        print(f"{m} OK")
    except anthropic.PermissionDeniedError:
        print(f"{m} 403 not enabled on this account")
EOF
```

Failed requests are not billed, so this costs nothing.

The runs

Every runner takes `--model haiku|sonnet`, `--mode <name>` for repeat runs, and `--dry-run` to build the prompts and print their token counts without calling anything.

stage	command	\$/run	min/run
B1 one-shot	<code>python tools/run_oneshot.py --all --model haiku</code>	0.11	3.8
B1 on Sonnet	<code>python tools/run_oneshot.py --all --model sonnet</code>	0.25	4.4
I1 + plan	<code>python tools/run_oneshot.py --all --with-plan --model haiku</code>	0.13	3.2
I2 + scanner	<code>python tools/run_oneshot.py --all --with-scanner --model haiku</code>	0.11	3.0
I3 verification	<code>python tools/verify_citations.py --from oneshot-haiku</code>	0.00	<0.1
I4 + cost	<code>python tools/run_oneshot.py --all --with-cost --model haiku</code>	0.11	3.0
I5 memory	<code>python tools/run_i5_memory.py --model haiku</code>	0.12	3.3
I6 multi-agent	<code>python tools/run_i6_multiagent.py --model haiku</code>	0.39	16.4
B2 agent + tools	<code>python tools/run_agent_b2.py --all --model haiku</code>	1.84	9.3

I2 requires checkov-scoped to have been run first; I1, I2 and I4 all require `./plan.sh --all`.

Score any of them with `python score.py --mode <mode>`, where the mode is printed at the end of each run.

Reproducing the variance check

This is the result that matters most, and it is the cheapest to reproduce:

```
for r in r2 r3 r4; do
    python tools/run_oneshot.py --all --model haiku --mode oneshot-haiku-$r
done
for m in oneshot-haiku oneshot-haiku-r2 oneshot-haiku-r3 oneshot-haiku-r4; do
    python score.py --mode $m | grep ^precision
```

done

~12 minutes, \$0.33. Expect four different numbers spanning roughly 0.73–0.90. That spread is the point: it is wider than five of the eight effects this project set out to measure, and it is why four of the original findings were withdrawn.

To reproduce the confirmed regressions, repeat I1, I6 and B2 three times each with `--mode <stage>-r2|r3|r4`. **~80 minutes, ~\$7.60.**

What to expect

Numbers marked (n=4) are means over four runs. Single runs should land within roughly ± 0.08 of the figures below — that is the measured noise floor, not a guess.

configuration	F1	\$/PR
Raw Checkov	0.052	0
Checkov scoped to the change	0.320	0
One prompt, Haiku 4.5 (n=4)	0.784 $\pm$ 0.078	0.007
One prompt, Sonnet 4.6	0.783	0.015
+ terraform plan (n=4)	0.674 $\pm$ 0.038	0.009
+ multi-agent split (n=4)	0.653 $\pm$ 0.060	0.026
+ tools, free exploration (n=4)	0.588 $\pm$ 0.042	0.133
+ scanner output	0.842	0.007
+ citation verification	0.900	0.000
+ cost table	0.842	0.007
+ review memory	0.800	0.007

Model outputs are not deterministic, so individual findings will differ between your run and ours. The three regressions should reproduce; the four no-measurable-effect stages are single runs inside the noise floor and may land either side of the baseline on any given attempt.

Total to reproduce everything: about 3 hours and \$10.44 — that is what this project actually spent, taken from the per-run costs recorded in `results/trajectories/`.

The short path

Four minutes, \$0.11, one command each. Enough to see the main result.

```
pip install -r requirements.txt
./plan.sh --all
python tools/run_oneshot.py --all --model haiku # the winner
python tools/run_oneshot.py --all --with-plan --model haiku # the plan makes it worse
python score.py --mode oneshot-haiku
python score.py --mode i1plan-haiku
```

Expect roughly 0.78 for the first and 0.67 for the second. The project's central hypothesis was that the second would beat the first.

About the state fixture

`terraform plan` against an empty state reports every resource as *create*, so a *replace* or a *destroy* cannot occur — and those are exactly the transitions a static scanner cannot see. Rather than standing up `LocalStack`, `tools/make_state.py` derives state from the plan's own `planned_values`, fills the identifiers that are only known after apply, and converges: write state, re-plan, fold the result back in, repeat until the plan is a clean no-op. It converges in four rounds.

The result is committed as `fixtures/base.tfstate`. **You do not need to run this** — it is included for completeness:

```
python tools/make_state.py --dir infra/base --out fixtures/base.tfstate
```

One consequence, documented because it is a real limitation: `aws_db_instance` always reports an in-place update against synthesized state even when every attribute before and after is identical. `tools/planfilter.py` drops updates whose before equals after. The rule is narrow and provable — an update that changes nothing is not a change — and it cannot hide a real finding, including the replace in case 08, which does change an attribute.

Troubleshooting

symptom	cause
403 ... is not available for this account	Bedrock model access not granted for that model in that region
404 on a model id	using a dated id, or the Mantle endpoint. See the three notes above
429 Too many requests	Bedrock throttling on multi-step stages. The client retries 8 times; <code>run_agent_b2.py</code> also resumes, skipping cases that already have output
bad interpreter: /usr/bin/env bash^M	line endings. <code>.gitattributes</code> pins LF; re-clone rather than converting
<code>.work/<case></code> missing	run <code>./plan.sh --all first</code>
checkov reports nothing	it needs the <code>.tf</code> files in <code>.work/<case>/</code> , which <code>plan.sh</code> creates